

# 实验 1 不同类型复杂网络的构建及读取

## 一、实验目的

- 1、学会构建不同类型的复杂网络
- 2、学会读入本地网络
- 3、学会将构建的网络保存到本地

## 二、实验内容

### 实验1-1 无向网络的生成

#### 无向无权网络的生成

```
In [9]: #利用add_nodes_from()以及add_edges_from()函数生成无向无权网络
import networkx as nx
import matplotlib.pyplot as plt
G=nx.Graph()
G.add_nodes_from([1,2,3,4,5])
G.add_edges_from([(1,2),(2,3),(2,5),(3,4),(4,5)])
As=nx.adjacency_matrix(G)
print("*****")
print(As) #输出邻接矩阵
print("*****")
print(As.todense()) #转换为二维数组形式的矩阵
nx.draw(G,node_size=500,with_labels=True,node_color="green", edge_color="red")
plt.show()
```

```
#利用利用add_edge()函数生成无向无权网络
import networkx as nx
import matplotlib.pyplot as plt
G=nx.Graph()
G.add_edge(1,2)
G.add_edge(1,3)
G.add_edge(1,4)
G.add_edge(2,3)
G.add_edge(3,4)
G.add_edge(4,5)
G.add_edge(4,6)
G.add_edge(5,6)
G.add_edge(5,7)
G.add_edge(5,8)
G.add_edge(6,7)
G.add_edge(6,8)
G.add_edge(7,8)
G.add_edge(7,9)
nx.draw(G,node_color="green", edge_color="red", with_labels = True, node_size = 500)
plt.show()
```

```
: #利用列表生成无向无权网络
import networkx as nx
import matplotlib.pyplot as plt
G=nx.Graph()
elist = [(1, 2), (2, 3), (1, 4), (4, 2)]
G.add_edges_from(elist)
nx.draw(G,node_color="green", edge_color="red", with_labels = True, node_size = 500)
plt.show()
```

```
: #已知邻接矩阵，创建无向无权网络
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
A=np.array([[0,1,1],[1,0,1],[1,1,0]]) #邻接矩阵
G=nx.from_numpy_array(A)
nx.draw(G,node_color="green", edge_color="red", with_labels = True, node_size = 500)
plt.show()
```

## 无向加权网络的生成

```
#生成无向加权网络
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd
G=nx.Graph()
G.add_nodes_from([1,2,3,4,5,6])
G.add_weighted_edges_from([(1,2,3), (1,3,1), (2,4,4), (2,5,1.5), (3,5,2), (3,6,3.5), (4,5,2)])
print("-----")
print(nx.to_pandas_edgelist(G))#输出连边以及对应的权重
print("-----")
w=[G[e[0]][e[1]]['weight'] for e in G.edges()]
nx.draw(G,node_color="green", edge_color="red",width=w,node_size=500,with_labels=True)
plt.show()
```

## 实验1-2 有向网络的生成

```
import networkx as nx
import matplotlib.pyplot as plt
DG=nx.DiGraph()
DG.add_nodes_from([1,2,3,4,5,6])
DG.add_edges_from([(1,2), (1,3), (2,4), (3,5), (4,6), (5,2), (5,4), (5,3), (3,6)])
nx.draw(DG,node_color="green", edge_color="red", with_labels = True, node_size = 500)
plt.show()
```

```
#有向加权网络
import networkx as nx
import matplotlib.pyplot as plt
DG=nx.DiGraph()
DG.add_nodes_from([1,2,3,4,5,6])
DG.add_weighted_edges_from([(1,2,2), (1,3,5), (2,4,2), (3,5,1), (4,6,3), (5,2,8), (5,4,2), (5,3,1), (3,6,2)])
w=[DG[e[0]][e[1]]['weight'] for e in DG.edges()]
nx.draw(DG,node_color="green", edge_color="red",width=w,node_size=500,with_labels=True)
plt.show()
```

```
import matplotlib.pyplot as plt
import networkx as nx
G = nx.DiGraph()
G.add_edge("a", "b", weight=0.6)
G.add_edge("a", "c", weight=0.2)
G.add_edge("c", "d", weight=0.1)
G.add_edge("c", "e", weight=0.7)
G.add_edge("c", "f", weight=0.9)
G.add_edge("a", "d", weight=0.3)

elarge = [(u, v) for (u, v, d) in G.edges(data=True) if d["weight"] > 0.5]
esmall = [(u, v) for (u, v, d) in G.edges(data=True) if d["weight"] <= 0.5]

pos = nx.spring_layout(G, seed=7) # positions for all nodes - seed for reproducibility

# nodes
nx.draw_networkx_nodes(G, pos, node_size=700)

# edges
nx.draw_networkx_edges(G, pos, edgelist=elarge, width=6)
nx.draw_networkx_edges(G, pos, edgelist=esmall, width=6, alpha=0.5, edge_color="b", style="dashed")

# node labels
nx.draw_networkx_labels(G, pos, font_size=20, font_family="sans-serif")
# edge weight labels
edge_labels = nx.get_edge_attributes(G, "weight")
nx.draw_networkx_edge_labels(G, pos, edge_labels)

ax = plt.gca()
ax.margins(0.08)
plt.axis("off")
plt.tight_layout()
plt.show()
```

## 实验1-3 利用库函数生成规则网络

```
#创建孤立节点图
import networkx as nx
import matplotlib.pyplot as plt
n=10
G=nx.Graph()
G.add_nodes_from(list(range(n)))
plt.figure(figsize=(4,4))
nx.draw(G,pos=nx.circular_layout(G),node_size=500,node_color="red",with_labels=True)
```

```
: #创建完全子图
import networkx as nx
import matplotlib.pyplot as plt
n=10
G=nx.complete_graph(n)
plt.figure(figsize=(4,4))
nx.draw(G,pos=nx.circular_layout(G),node_size=500,node_color="red",with_labels=True)
plt.savefig("完全子图.pdf")
```

...

```
: #创建一维环状图
import networkx as nx
import matplotlib.pyplot as plt
n=10
G=nx.cycle_graph(n)
plt.figure(figsize=(4,4))
nx.draw(G,pos=nx.circular_layout(G),node_size=500,node_color="red",with_labels=True)
plt.savefig("一维环状图.pdf")
```

...

```
: #K近邻耦合图
import networkx as nx
import matplotlib.pyplot as plt
n=10
G=nx.watts_strogatz_graph(n,4,0.0)
plt.figure(figsize=(4,4))
nx.draw(G,pos=nx.circular_layout(G),node_size=500,node_color="red",with_labels=True)
plt.savefig("K近邻耦合图.pdf")
```

...

```
: #二维方格图
import networkx as nx
import matplotlib.pyplot as plt
G=nx.grid_graph((6,6),periodic=False)
plt.figure(figsize=(4,4))
nx.draw(G,node_size=50,node_color="red",with_labels=False)
plt.savefig("二维方格图.pdf")
```

```

#星型网络
import networkx as nx
import matplotlib.pyplot as plt
G=nx.star_graph(10)
plt.figure(figsize=(4,4))
nx.draw(G,node_size=50,node_color="red",with_labels=False)
plt.savefig("星型网络.pdf")

```

...

```

#karate_club
import matplotlib.pyplot as plt
import networkx as nx

G = nx.karate_club_graph()
print("Node Degree")
for v in G:
    print(f"{v:4} {G.degree(v):6}")

nx.draw_circular(G, with_labels=True)
plt.show()

```

## 实验1-4 从本地导入网络

```

# 加载真实数据集，文件为.csv
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("d:\\power.csv")
G = nx.from_pandas_edgelist(df, 'source', 'target', create_using = nx.Graph())
d = dict(nx.degree(G))
print("网络平均度为: ", sum(d.values())/len(G.nodes))
nx.draw(G,node_color="green", edge_color="red", with_labels = True, node_size = 50)
plt.title("power network",fontsize=20)
plt.show()

```

```
# 加载真实数据集, 文件为.txt
import networkx as nx
import pandas as pd
df = pd.read_table("d:\\power.txt")
G = nx.from_pandas_edgelist(df, 'source', 'target', create_using = nx.Graph())
d = dict(nx.degree(G))
print("网络平均度为: ", sum(d.values())/len(G.nodes))
```

```
# 加载真实数据集, 有权重
import networkx as nx
import pandas as pd
df = pd.read_csv("d:\\actor_edges.csv")
G = nx.from_pandas_edgelist(df, 'from', 'to', 'weight', create_using = nx.Graph())
d = dict(nx.degree(G))
print("网络平均度为: ", sum(d.values())/len(G.nodes))
```

```
# 加载真实数据集, 邻接矩阵
import networkx as nx
import pandas as pd
df = pd.read_csv("d:\\adjacency_matrix.csv", header=None, index_col=None)
G = nx.from_pandas_adjacency(df)
d = dict(nx.degree(G))
print("网络平均度为: ", sum(d.values())/len(G.nodes))
```

## 实验1-5 网络数据保存到本地

```
#无权网络保存到本地文件为csv格式
import networkx as nx
import pandas as pd
BA=nx.barabasi_albert_graph(200,2,3)
source=[]
target=[]
for e in BA.edges():
    source.append(e[0])
    target.append(e[1])

df1 = pd.DataFrame({'source': source, 'target': target})
df1.to_csv('d:\\BA.csv')
```

```
#加权网络保存到本地, 文件为csv格式
import networkx as nx
import pandas as pd
G = nx.Graph()
G.add_nodes_from([1,2,3,4,5,6])
G.add_weighted_edges_from([(1,2,3), (1,3,1), (2,4,4), (2,5,1.5), (3,5,2), (3,6,3.5), (4,5,2.5), (4,6,0.5)])

source=[]
target=[]
for e in G.edges():
    source.append(e[0])
    target.append(e[1])

w=[G[e[0]][e[1]]['weight'] for e in G.edges()]
df1 = pd.DataFrame({'source': source, 'target': target, 'weight':w})
df1.to_csv('d:\\G.csv')
```

## 三、实验报告内容



实验1-学号-姓名.  
docx